

tf.tensor_scatter_nd_add Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/tensor_scatter_nd_add

Adds sparse updates to an existing tensor according to indices.

Function Signatures

```
tf.tensor_scatter_nd_add( tensor: Annotated[Any,
TV_TensorScatterAdd_T], indices: Annotated[Any,
TV_TensorScatterAdd_Tindices], updates: Annotated[Any,
TV_TensorScatterAdd_T], name=None ) -> Annotated[Any,
TV_TensorScatterAdd_T]
```

Code Examples

```
tf.tensor_scatter_nd_add(
    tensor: Annotated[Any, TV_TensorScatterAdd_T],
    indices: Annotated[Any, TV_TensorScatterAdd_Tindices],
    updates: Annotated[Any, TV_TensorScatterAdd_T],
    name=None
) -> Annotated[Any, TV_TensorScatterAdd_T]
```

```
tf.tensor_scatter_nd_add(  
    tensor: Annotated[Any, TV_TensorScatterAdd_T],  
    indices: Annotated[Any, TV_TensorScatterAdd_Tindices],  
    updates: Annotated[Any, TV_TensorScatterAdd_T],  
    name=None  
) -> Annotated[Any, TV_TensorScatterAdd_T]
```

```
indices.shape[-1] <= tensor.shape.rank
```

```
indices.shape[-1] <= tensor.shape.rank
```

```
indices.shape[:-1] + tensor.shape[indices.shape[-1]:]
```

```
indices.shape[:-1] + tensor.shape[indices.shape[-1]:]
```

```
indices = tf.constant([[4], [3], [1], [7]])  
updates = tf.constant([9, 10, 11, 12])  
tensor = tf.ones([8], dtype=tf.int32)  
updated = tf.tensor_scatter_nd_add(tensor, indices, updates)  
updated
```

```
indices = tf.constant([[4], [3], [1], [7]])
```

Documentation

Adds sparse updates to an existing tensor according to indices.

View aliases

Compat aliases for migration

See

Migration guide for
more details.

`tf.compat.v1.tensor_scatter_add`, `tf.compat.v1.tensor_scatter_nd_add`

Used in the notebooks

- Extension types
 - Introduction to tensor slicing

This operation creates a new tensor by adding sparse updates to the passed in tensor.

This operation is very similar to `tf.compat.v1.scatter_nd_add`, except that the updates are added onto an existing tensor (as opposed to a variable). If the memory for the existing tensor cannot be re-used, a copy is made and updated.

`indices` is an integer tensor containing indices into a new tensor of shape `tensor.shape`. The last dimension of `indices` can be at most the rank of `tensor.shape`:

The last dimension of `indices` corresponds to indices into elements (if `indices.shape[-1] = tensor.shape.rank`) or slices (if `indices.shape[-1] < tensor.shape.rank`) along dimension `indices.shape[-1]` of `tensor.shape`. `updates` is a tensor with shape

The simplest form of `tensor_scatter_nd_add` is to add individual elements to a tensor by index. For example, say we want to add 4 elements in a rank-1 tensor with 8 elements.

In Python, this scatter add operation would look like this:

We can also, insert entire slices of a higher rank tensor all at once. For example, if we wanted to insert two slices in the first dimension of a rank-3 tensor with two matrices of new values.

In Python, this scatter add operation would look like this:

Returns